Security & Operational Risk Analysis — Check-In/Check-Out System

> Generated from a full manual review of the source code (not the `docs/` folder — docs are out of date). Review date: 2026-08-31 · Repo: `checkinout82026-cmd/Checkinout-ai` (public GitHub) · HEAD: `f9c8f90`

---

0. Executive Summary

This is a **child-safety attendance system handling real PII of minors**, and it is currently **not safe to run in production**. The three most urgent items:

1. **Real student/parent PII (~100 real children's names + parent phone numbers) is committed to a public GitHub repository** in `actual_students.csv` and hardcoded in `src/lib/seedData.ts`.
2. **The Firestore database is open to the entire world** (`allow read, write: if true` on every collection) and the Firebase project config/API key is public — anyone can read, modify, or wipe all data, including plaintext staff passwords.
3. **Authentication is trivially bypassable** — a hardcoded master password, a universal `password` fallback, client-side-only role checks, and an auth path that succeeds even when Firebase Auth rejects the login.

---

1. CRITICAL Risks

C1. Hardcoded plaintext admin/staff credentials committed to a public repo

- **Where:** `src/lib/db.ts` lines 20–47 (`defaultUsers`)
  - Admin: username `KeshavKousik`, password `Giridharan#20`, email `keshavkousik@school.com`
  - Staff: username `adams.rachel`, password `Password123!`
- **Also re-hardcoded as a master-key check in** `src/lib/auth.ts` lines 104–110 (`password === 'Giridharan#20'` accepted even if the stored password differs).
- **Risk:** Anyone on the internet can log in as **administrator**. `Giridharan#20` looks like a real personal password pattern (name + birth year) — if reused personally, it is now public. These credentials are also in **git history**, so removing them from HEAD is not enough.
- **Fix:** Rotate the admin password immediately (both app and any personal reuse). Remove all hardcoded credentials from code and history (BFG/git-filter-repo + force push). Enforce Firebase Auth–only sign-in.

C2. Universal password backdoor — any known username logs in with `password`

- **Where:** `src/lib/auth.ts` line 109: `(password === 'password')` inside

`isPasswordCorrect`.
- **Also:** `src/lib/db.ts` lines 82, 149 (`password: u.password || 'password'`) and `src/components/AdminStaff.tsx` lines 56, 137 (`password.trim() || 'password'`) — new/edited accounts with a blank password field get the literal password `password`.
- **Risk:** Username list is readable by anyone (open Firestore rules + it's shown in the staff-approval dropdown). Any account → admin dashboard. Password reset emails guess `${username}@school.org` (`src/lib/auth.ts` line 220), enabling account takeover via Firebase reset for guessed emails.
- **Fix:** Delete the backdoor; require strong passwords at account creation; never default passwords.

C3. Firestore database is world-readable/world-writable

- **Where:** `firestore.rules` — every collection (`users`, `students`, `authorized_pickups`, `attendance`) has `allow read, write: if true;`.
- **Compounding factor:** `firebase-applet-config.json` (projectId `gen-lang-client-0658931070`, API key `AIzaSyA4npHqr...`, non-default database ID `ai-studio-remixremixchecki-...`) is **committed to the public repo**. `recaptchaSiteKey` is empty → **no Firebase App Check**.
- **Risk:** Complete anonymous compromise: read all student PII and plaintext staff passwords; forge attendance records; edit any user doc to `role: 'admin'`; delete the entire database. No rate limiting or abuse protection exists server-side.
- **Fix:** Deploy authenticated, least-privilege rules; enable App Check; move the config out of the repo and treat the current data as breached (rotate everything, scrub or replace the database).

C4. Real PII of minors committed to source control

- **Where:**
  - `actual_students.csv` — **103 real students**: real full names, student IDs (Kumon card numbers), home phone numbers, mother's/father's names and cell phone numbers.
  - `src/lib/seedData.ts` (~3,540 lines, `ACTUAL_STUDENTS`) — the same real PII hardcoded as TypeScript, exported as `TEN_STUDENTS`/`generate10Students` and **auto-seeded into Firestore** on every app start (`src/lib/db.ts` `db.init()` lines 715–774).
- **Risk:** Privacy/data-protection violation for children's data (COPPA / FERPA-adjacent exposure; school liability). Phone numbers enable targeted harassment of families. Data is duplicated in three places (CSV, seed file, Firestore) and lives in git history.
- **Fix:** Purge both files from repo **and git history**; replace with synthetic seed data; notify stakeholders; treat the dataset as disclosed; establish a policy that PII never enters the repo.

C5. Plaintext passwords stored and synced everywhere

- **Where:** `src/lib/db.ts` `saveUsers`/`saveUser` write `password` in plaintext to Firestore `users` docs (lines 82, 149); `loadUsersFromFirestore` pulls **all users with passwords** into every browser and persists them in `localStorage['checkin_users']` (lines 60–70, 99–128, 131–162).
- **Risk:** With C3, passwords are readable by anyone; on shared kiosk machines any user of the browser can extract staff credentials from localStorage/DevTools.
- **Fix:** Never store passwords outside Firebase Auth; strip `password` from any client-read data; clear localStorage of credential-bearing

caches.

C6. Staff approval for releasing a child requires no verification at all

- **Where:** `src/components/StaffApprovalModal.tsx` `handleVerifyAndApprove` (lines 49–61) — the "authorization" is just **picking a staff name from a dropdown**; no password, no PIN, no re-auth. (The comment in `StudentDashboard.tsx` line 127 falsely claims "ONLY after staff enters valid password".)
- **Risk:** Anyone standing at the kiosk (or anyone with the open database + app) can check a child out to a pickup person of their choosing. This defeats the core child-safety control of the product.
- **Fix:** Require re-authentication (Firebase Auth reauth or PIN) validated server-side; log the approver's verified identity.

---

2. HIGH Risks

H1. Authentication succeeds even when Firebase Auth fails (client-side trust)

- **Where:** `src/lib/auth.ts` lines 111–123 — after the plaintext match, `signInWithEmailAndPassword` failure is swallowed (`catch { return match; }`) and the user is logged in anyway. The session is a JSON blob in `localStorage['activeUser']` (`src/App.tsx` lines 28–41, 56–66) that is **fully trusted on load with no server validation**.
- **Risk:** Disabled/deleted/stale accounts still authenticate; tampering with `activeUser` in DevTools grants any role, including admin. All "authorization" is UI-level only.

H2. Role assigned by email string matching — automatic privilege escalation paths

- **Where:** `src/lib/auth.ts` lines 35, 44, 69 — `role: email.toLowerCase().includes('admin') ? 'admin' : 'staff'`.
- **Also:** `signInWithGoogle(defaultRole = 'staff')` (lines 197–209) — **any** Google account that completes the popup is written into `users` as `staff` (`updateDoc(... { role: defaultRole })`), and an existing user is matched merely by username == gmail local-part (line 42) → identity confusion/impersonation of real staff.
- **Risk:** Arbitrary Google accounts gain staff dashboards; anyone can register an email containing "admin" and self-escalate.

H3. No real identity verification for check-in / impersonation of students

- **Where:** `src/components/CheckInOut.tsx` — check-in needs only a typed name/ID with live name-suggestion autocomplete (lines 52–103); no PIN, card, or photo verification. `StudentDashboard.tsx` identifies students by `user.id` alone.
- **Risk:** One child can be checked in/out as another; the pickup person is free-text selectable with a "custom" option. Combined with C6, the entire attendance record is advisory, not controlled.

H4. SMS notifications are simulated — parents get false assurance

- **Where:** No SMS provider anywhere in the code. `smsNotificationSent` / `smsSentAt` are just booleans/timestamps

written to Firestore (`src/components/StudentDashboard.tsx` line 100, `CheckInOut.tsx`, `firebase-blueprint.json`).
- **Risk:** In a real incident (child not picked up / custody dispute), records claim "SMS sent" when no message existed. **Child-safety operational failure.** Additionally, "custom phone" targets are free-text with no validation.
- **Fix:** Integrate a real provider with delivery receipts, or remove the feature/labels until it exists.

H5. Destructive auto-seeding & unguarded data scripts (data-loss risk)

- **Where:**
  - `src/lib/db.ts` `db.init()` (lines 734–752): if the Firestore `students` collection is **empty OR has ≤ 10 docs OR contains certain names**, it **deletes every existing student doc** and re-seeds from the hardcoded roster. A small legitimate roster gets wiped on next app load.
  - `clear_data.js`, `clear_data2.js`, `seed_firestore.ts`, `migrate_attendance.ts`, `migrate_pickups.ts` — delete/overwrite whole collections with **no confirmation or safety checks**; `clear_data.js` even targets the **default** database (line 7) while the others use the custom `firestoreDatabaseId` — scripts may silently act on the wrong database.
  - `db.init()` also **force-writes the hardcoded admin (`admin_keshav`) back into Firestore** on every load (lines 108–119), resurrecting C1's credential even if an operator deletes it.
- **Fix:** Gate all seeding/migration behind explicit, confirm-protected scripts; remove auto-destruct logic from app startup; standardize the database ID.

H6. Session/PII persistence on shared kiosk devices

- **Where:** `src/App.tsx` + `src/lib/db.ts` — the full student roster, attendance, and the **users list with plaintext passwords** are cached in localStorage (`checkin_users`, `checkin_students`, `checkin_attendance`); the `activeUser` session persists across browser restarts with **no expiry**; logout only clears `activeUser`/`appMode`, not the data caches.
- **Risk:** On a shared/public kiosk, subsequent users (or anyone with device access) can view the whole roster, parent phone numbers, and staff credentials.

---

3. MEDIUM Risks

M1. Username enumeration & no rate limiting on the app's own path

- Login distinguishes "Incorrect password" vs "Username not found" (`src/lib/auth.ts` lines 122, 134–139); password reset guesses `${username}@school.org`. The plaintext password check happens **before** Firebase Auth, bypassing Firebase's built-in throttling entirely.

M2. Forgeable attendance data / no server-side integrity

- All writes go straight from the browser to the open database: attendance records, pickup persons, timestamps are client-supplied. `checkInStaffId`/`checkOutStaffId` record the kiosk's logged-

in user, not who physically acted — no audit trail worth the name. No server timestamps, no immutability, no logs.

M3. Unused/over-provisioned dependencies and platform claims

- `package.json` includes `express`, `dotenv`, `@google/genai`, `react-router-dom`, `motion` — unused in the app; `metadata.json` claims `MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API` which does not exist in code. Extra supply-chain/attack surface and misleading deployment assumptions. `npm run dev` binds `0.0.0.0` (`--host=0.0.0.0`) — dev server exposed on the LAN.

M4. Config drift and secrets-in-config hygiene

- Firebase config read from committed JSON (`src/lib/firebase.ts` line 4) instead of environment variables; `.env.example` references a Gemini key/APP_URL that the app never uses — misleads reviewers into thinking secrets are externalized. Two lockfiles (`package-lock.json` + `bun.lock`) risk inconsistent dependency resolution.

M5. Input handling / content injection surface

- CSV import accepts arbitrary pasted text and maps columns by fuzzy header matching (`src/lib/csvParser.ts`); malformed input silently fabricates students with default phone `555-0100` and guessed IDs (`1000${i}`) — can pollute the roster. Names from imports are interpolated into toasts/UI; student IDs are generated with `Math.random()` (predictable, collision-prone).

M6. PII over-exposure in UI

- The kiosk search surfaces students' full names as autocomplete while typing; parent phone numbers and "notes" containing home phone numbers are displayed in management screens to any staff-level session (trivially obtained — see C2/H1/H2).

---

4. LOW Risks

- **L1.** `measurementId` empty but analytics plumbing present; `oAuthClientId` committed (low sensitivity, but part of the public footprint).
- **L2.** Error handling dumps internal warnings to `console.warn` (e.g., Firestore errors with project info) on kiosk devices.
- **L3.** `formatPhoneNumber` / `convertTimeToIso` silently mangle odd inputs — data-quality issues in a safety-critical log.
- **L4.** No CSP/security headers configured anywhere in the repo (Vite-only static app; deployment hardening undocumented and unverified).
- **L5.** `AdminStaff` allows admins to view/set plaintext passwords in the edit form (`user.password || ''` prefilled) — normalizes credential handling that should not exist.

5. Credential & Data Inventory (what's actually in the repo today)

Item	Location	Exposure

Admin username/password (`KeshavKousik` / `Giridharan#20`)	`src/lib/db.ts:24`, `src/lib/auth.ts:107–108`	Public (repo + git history)
Staff password (`Password123!`)	`src/lib/db.ts:37`	Public (repo + git history)
Universal password `password`	`src/lib/auth.ts:109`, `db.ts:82,149`, `AdminStaff.tsx:56,137`	Public (source)
Firebase API key + projectId + appId + non-default DB id	`firebase-applet-config.json`	Public (repo)
OAuth client ID	`firebase-applet-config.json:10`	Public (repo)
Real student PII: 103 children, IDs, parent names, ~200 phone numbers	`actual_students.csv`, `src/lib/seedData.ts`	Public (repo + git history) + auto-seeded to open Firestore
Plaintext staff passwords	Firestore `users` collection + `localStorage['checkin_users']` on every client	World-readable (open rules)
Open Firestore rules	`firestore.rules`	Whole DB publicly writable

---

6. Prioritized Remediation Plan

1. **Immediately rotate** the admin (`Giridharan#20`) and staff (`Password123!`) passwords; check personal reuse of `Giridharan#20`.
2. **Purge PII and credentials from git history** (BFG Repo-Cleaner / `git filter-repo`) and force-push; contact GitHub support to clear cached views if needed.
3. **Lock down Firestore rules** (auth-required, per-role least privilege) and **enable App Check**; then scrub/rotate the current database contents.
4. **Delete the auth backdoors**: the `password === 'password'` fallback, hardcoded `Giridharan#20` checks, and the “return match anyway” catch in `signInWithEmail`. Enforce Firebase Auth as the only credential verifier.
5. **Add real verification to staff approval** (re-auth/PIN, validated server-side) and log the verified approver.
6. **Strip `password` from all client data paths** and localStorage caches; cache only what the UI needs.
7. **Remove auto-seed/destructive logic** from `db.init()`; protect migration scripts with explicit confirmation and a single, correct database ID.
8. **Replace simulated SMS** with a real provider + delivery status, or disable the claims.
9. Move Firebase config to environment/config injection; remove unused deps and fix `metadata.json`.
10. Add rate limiting/lockout, non-enumerating error messages, session expiry, and (for kiosks) auto-lock + cache clearing.

Every item above was verified directly against the current source files at commit `f9c8f90`; file/line references are included for audit.